

All four Impinj Gen2X features verified working - FXR90 app 5.0.7

Every Gen2X feature was tested for **measured effect on real tags**, not just for an HTTP 200. All four work.

Reader	10.233.48.49 - FXR90, serial 25289523070378, app 5.0.7
Antennas	5 and 6, 30 dBm
Date	2026-09-08
Tags	2 x Impinj Monza 4i (MDID 0x801, TMN 0x1B0)
Spec	openAPISpec 12.yaml
Result	[OK] fastID, tagFocus, tagQuieting (basic + advanced), tagProtect - all confirmed

Target **e28011b0a5050076c4d7530a** - control **e28011b0a505007698ca0895**.

1. Summary

Feature	Accept	Measured effect	Verdict
fastID	[OK] 200	EPC+TID returned, format: "fastID"	[OK] works
tagFocus	[OK] 200	16 -> 2 reads, 3/3 trials	[OK] works
tagQuieting.ba	[OK] 200	target 8 -> 0 reads, control 8 throughout	[OK] works
tagQuieting.ad	[OK] 200	stored verbatim; no effect on non-matching tags (correct)	[OK] accepted
tagProtect	[OK] 200	5/5 -> 0/5 -> 5/5	[OK] works - see ../tagprotect-REVERIFIED-507/
Mutual exclusion	[OK] 422	two features in one request rejected	[OK] enforced as documented

2. [OK] fastID - returns the TID inline

```
PUT /cloud/impinjGen2X {"fastID":{"enabled":true}}
PUT /cloud/start {"applyImpinjGen2X":true}
```

A plain inventory - **no `accesses` block** - returned:

```
{"data":{"antenna":5,"eventNum":95,"format":"fastID",
  "idHex":"e28011b0a5050076c4d7530ae28011b02000b30a26ba03b6",
  "peakRssi":-24},
  "timestamp":"2026-09-08T09:25:54.162+0000","type":"CUSTOM"}
```

idHex is **48 hex chars** - **EPC concatenated with TID**, and **format** changes from **"epc"** to **"fastID"**:

Tag	EPC (chars 0-23)	TID (chars 24-47)	Decodes to
target	e28011b0a5050076c4d7530a	e28011b02000b30a26ba03b6	Monza 4i
control	e28011b0a505007698ca0895	e28011b02000a895c65003b4	Monza 4i

Both TIDs are **byte-identical** to the TIDs read separately via `accesses: [{type:READ, membank:TID}]`, which cross-validates the feature.

This resolves an open item from earlier testing. A previous report recorded "FastID produces no TID". That was a reading error on my part - the TID is appended to `idHex`, not placed in a separate field, and `format` flips to `fastID` to signal it. There is no defect.

Worth documenting: consumers must parse `idHex` by length, or key off `format == "fastID"`. A client that assumes `idHex` is always the EPC will read a 48-character value as one long EPC.

Capture: `evidence/fastid.ndjson`

3. [OK] tagFocus - 16 reads down to 2

TagFocus keeps a tag in session S1 so it replies **once** per inventory instead of repeatedly. Measuring it requires raw per-read reporting, otherwise the reader's own de-duplication hides the effect:

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[5,6],"transmitPower":[30,30],
 "reportFilter":{"duration":0,"type":"PER_ANTENNA"},
 "tagMetadata":["ANTENNA","RSSI","SEEN_COUNT"]}
```

`reportFilter.duration: 0` reports every individual read. Then, 12-second inventories:

Trial	tagFocus:false	tagFocus:true
1	16 reads (8 per tag)	2 reads (1 per tag)
2	16	2
3	16	2

Zero variance across three A/B pairs. Exactly one read per tag with the feature on, eight without.

Method note. The first attempt at this test used the default report filter and produced 1 read per tag in *both* states - the effect was completely masked. Anyone documenting or testing TagFocus needs `reportFilter.duration = 0`, or the feature looks like it does nothing.

Captures: `evidence/tf_false.ndjson`, `evidence/tf_true.ndjson`, plus `evidence/tfr_{true,false}_{1,2}.ndjson` for the repeat trials.

4. [OK] tagQuieting.basic - target silenced, control untouched

```
PUT /cloud/impinjGen2X
{"tagQuieting":{"basic":{"action":"quiet",
                        "tagIDs":["e28011b0a5050076c4d7530a"]}}}
PUT /cloud/start {"applyImpinjGen2X":true}
```

Raw per-read reporting, 12 s per run:

Phase	Target reads	Control reads
Baseline	8	8
quiet run 1	0	8
quiet run 2	0	8
quiet run 3	0	8

unquiet run 1	8	8
unquiet run 2	8	8
unquiet run 3	8	8

The control tag held at **exactly 8 reads in all seven runs**, isolating the effect to the named EPC. Fully reversible.

Captures: `evidence/tq_base.ndjson`, `evidence/tq_q{1,2,3}.ndjson`, `evidence/tq_u{1,2,3}.ndjson`

5. [OK] tagQuieting.advanced - accepted and stored verbatim

The spec's `advanced_quiet_tags` example was sent unmodified - two `preSelect` entries, `tagQuietMasks: [SL_ASSERT, S2B]`, `target: SL`, `stateAwareAction: ASSERTSL_DEASSERTSL`. Returned **200** and read back byte-for-byte identical.

Its masks target EPCs `E280123456789012345678AB` and `E280AABBCCDDEEFF11223344`, neither of which is present in the field. With it applied, both real tags stayed at **8 reads each** - correct: a mask that matches nothing should quiet nothing.

So this is confirmed accepted and correctly scoped, but its quieting effect is not directly proven - that would need a tag whose EPC matches the mask. Flagged rather than claimed.

Payload: `evidence/tq_adv.json` - capture: `evidence/tq_adv_scan.ndjson`

6. [BUG] The tagQuieting error message is better than the spec

My first attempt used `{"tagQuieting":{"enabled":true,"mode":"basic"}}` - wrong, but the reader's reply was genuinely helpful:

```
{
  "code": 3,
  "message": "\"set_impinjGen2X\" Failed to apply. Description: unsuccessful rc response: Failure: Unknown tagQuieting field: 'enabled' (valid: basic, advanced)"}
}
```

It names the offending field **and** lists the valid alternatives. That is the clearest validation message seen anywhere in this API - worth holding up as the model for the vaguer ones elsewhere (`invalid "additional" JSON object, Invalid payload fields`).

Unlike `fastID` and `tagFocus`, `tagQuieting` takes **no `enabled` flag** - the feature is selected by supplying either a `basic` or an `advanced` object. That asymmetry between the four features is easy to miss from the schema alone.

7. [OK] Mutual exclusion is enforced

The spec states: *"Exactly one feature object ... must be provided per request; combining any two or more features is rejected."* Confirmed:

```
PUT /cloud/impinjGen2X
{"fastID":{"enabled":true},"tagFocus":{"enabled":true}}

-> 422 {"code":3,"message":"... Failure: Only one Impinj Gen2X feature can be configured at a time (fastID, tagProtect, tagFocus, or tagQuieting)"}
}
```

Behaviour and message both match the documentation.

8. Tag and reader state after testing

	Before	After	
Target reads (plain, raw filter)	8	8	[OK] restored
Control reads	8	8	[OK] untouched
Target access password	12348765	12348765	[OK] unchanged
Target kill password	12348765	12348765	[OK] unchanged
Control passwords	12348765	12348765	[OK] unchanged
impinjGen2X	{"fastID":{"enabled":false}}	same	[OK] idle
Reader mode	CUSTOM, ant [5,6], 30 dBm	same	[OK] restored
Data endpoint	WEBSOCKET_TEST	WEBSOCKET_TEST	[OK] restored

Every tag was unquieted and left readable. **Nothing was written to either tag's memory** - passwords were read only, verified unchanged at the end.

Final checks: [evidence/final.ndjson](#), [evidence/pw_final.ndjson](#)

9. What would improve the documentation

1. **Document the `fastID` output format.** `idHex` becomes EPC+TID (48 hex chars) and `format` becomes "`fastID`". Neither is stated, and a client parsing `idHex` as an EPC will silently mis-read it.
2. **Note that `tagFocus` needs `reportFilter.duration = 0` to observe.** With the default filter the feature appears to do nothing.
3. **Call out that `tagQuieting` has no `enabled` field,** unlike the other three features - it is selected by providing `basic` or `advanced`.
4. **`tagQuieting.advanced` has no worked result anywhere.** The example is syntactically fine but its semantics (which `tagQuietMasks` / `stateAwareAction` combinations do what) are undocumented.

10. Files

```
all-features-VERIFIED-507/
|-- GEN2X_ALL_FEATURES_507.md      <- this file
`-- evidence/
    |-- fastid.ndjson              EPC+TID inline
    |-- tf_false.ndjson tf_true.ndjson  TagFocus A/B trial 1
    |-- tfr_false_1/2.ndjson tfr_true_1/2.ndjson trials 2-3
    |-- tq_base.ndjson             tagQuieting baseline
    |-- tq_q1/q2/q3.ndjson         quieted
    |-- tq_u1/u2/u3.ndjson         unquieted
    |-- tq_adv.json tq_adv_scan.ndjson advanced example + scan
    |-- final.ndjson pw_final.ndjson  end-state checks
    |-- mode_raw.json              reportFilter duration 0 (the key payload)
    |-- mode_plain.json mode_pw.json
    `-- scan.sh                    one inventory + MQTT capture
```

Recount any result:

```
wc -l evidence/tf_false.ndjson evidence/tf_true.ndjson      # 16 vs 2
grep -c e28011b0a5050076c4d7530a evidence/tq_q*.ndjson    # target: all 0
grep -c e28011b0a505007698ca0895 evidence/tq_q*.ndjson    # control: all 8
```